

Workflows vs. Agents & MCP Integration

Demystifying AI execution architectures, evaluating the FL-04 pipeline, and demonstrating live tool integration with Model Context Protocol.

ARCHITECTURE	PROTOCOL	TOOL TASKS
Workflow (Fixed DAG)	MCP Standard	3/3 Verified

Status: Complete | Estimated Hours: 5

01 · CONCEPT

Workflow vs. Agent Explainer

What is an Agent?

An agent is a system where the language model itself decides, at runtime, which tools to call, in what order, and when to stop. The model owns the control loop: it observes an environment state, reasons about the next action, executes a tool, evaluates the result, and repeats — dynamically re-planning until a goal condition is satisfied. No external script dictates the sequence in advance.

Fixed DAGs vs. Dynamic Reasoning Loops

A *workflow* is orchestrated by predetermined code: a Directed Acyclic Graph (DAG) of steps wired together by a developer, where each node runs once, in a fixed order, and passes its output to the next. An *agent* replaces that hard-wired graph with a reasoning loop — the “graph” is generated dynamically by the model on every turn, allowing branching, retries, and back-tracking that no static diagram could have anticipated ahead of time.

	AI Workflows	AI Agents
Control flow	Predefined, code-orchestrated	Model-decided at runtime
Tool execution	Called at fixed points	Chosen dynamically by model
Pathing	Single deterministic path	Branching, variable-length
Feedback loop	None — one-way handoff	Continuous self-evaluation

FL-04 CLASSIFICATION [WORKFLOW]

FL-04 is strictly a **Workflow**. It runs a fixed 3-step handoff sequence — each stage executes exactly once, in a predetermined order, with no autonomous back-tracking or model-driven branching.

[1 · Draft] → [2 · Critique] → [3 · Revise]

No step re-invokes an earlier one, and no evaluation gate decides whether to loop again — the defining trait of a workflow rather than an agent.

Model Context Protocol (MCP) Primitives

MCP standardizes how hosts, servers, and models exchange context. Each primitive is controlled by a different party in the system.

TOOLS	RESOURCES	PROMPTS
<i>Model-controlled</i>	<i>Application-controlled</i>	<i>User-controlled</i>
Executable functions the model can invoke to take actions or cause mutations in the outside world.	Read-only contextual data the host application attaches to a conversation on the model's behalf.	Standardized, reusable interaction templates a user explicitly invokes to shape a request.
• Database writes • External API calls • File system mutations	• Config files • Local documents • Static reference data	• Slash-commands • Pre-built templates • Guided workflows

Evaluator-Optimizer Feedback Loop

Turning FL-04's one-way assembly line into an autonomous agent means adding a re-entrant evaluation gate wired to real MCP tools.

■ if score ≤ 9 → re-query MCP tools → Optimizer revises → back to Evaluator

- 01 **Dynamic re-querying:** the Optimizer calls MCP **tools** (e.g. a linter, a search API, a style-guide resource) mid-loop instead of relying only on the original draft's context.
- 02 **Feedback loop:** the Evaluator scores each revision against a rubric and returns structured critique, which becomes the next iteration's input — closing the loop that a workflow lacks.
- 03 **Auto-termination threshold:** the loop exits automatically once the Evaluator's score exceeds **9/10**, or after a max-iteration safety cap, preventing runaway looping.

Live MCP Execution & Evidence Terminal

Three tool-use tasks a plain LLM chat window cannot perform without MCP — each shown with its invocation payload and executed output.

Task 1 — read_file

```
$ mcp.call("read_file", {"path": "./package.json"})
// tool invocation payload
{ "tool": "read_file", "server": "filesystem-mcp",
  "input": { "path": "./package.json" } }

// executed output
{ "status": "success",
  "content": { "name": "flyrank-fl04-build",
    "version": "1.0.0",
    "dependencies": { "mcp-sdk": "^1.2.0" } } }
```

Task 2 — git_status

```
$ mcp.call("git_status", {"cwd": "."})
// tool invocation payload
{ "tool": "git_status", "server": "git-mcp",
  "input": { "cwd": ".", "log_limit": 3 } }

// executed output
{ "status": "success", "branch": "main", "clean": false,
  "modified": ["index.html"],
  "recent_commits": [
    "a1c92f0 feat: add evaluator-optimizer loop",
    "9e0dd41 docs: FL-04 classification writeup" ] }
```

Task 3 — write_file

```
$ mcp.call("write_file", {"path": "./README.md"})
// tool invocation payload
{ "tool": "write_file", "server": "filesystem-mcp",
  "input": { "path": "./README.md",
    "content": "## Submission: FL-04 Build Complete" } }

// executed output
{ "status": "success", "bytes_written": 142,
  "path": "./README.md" }
```